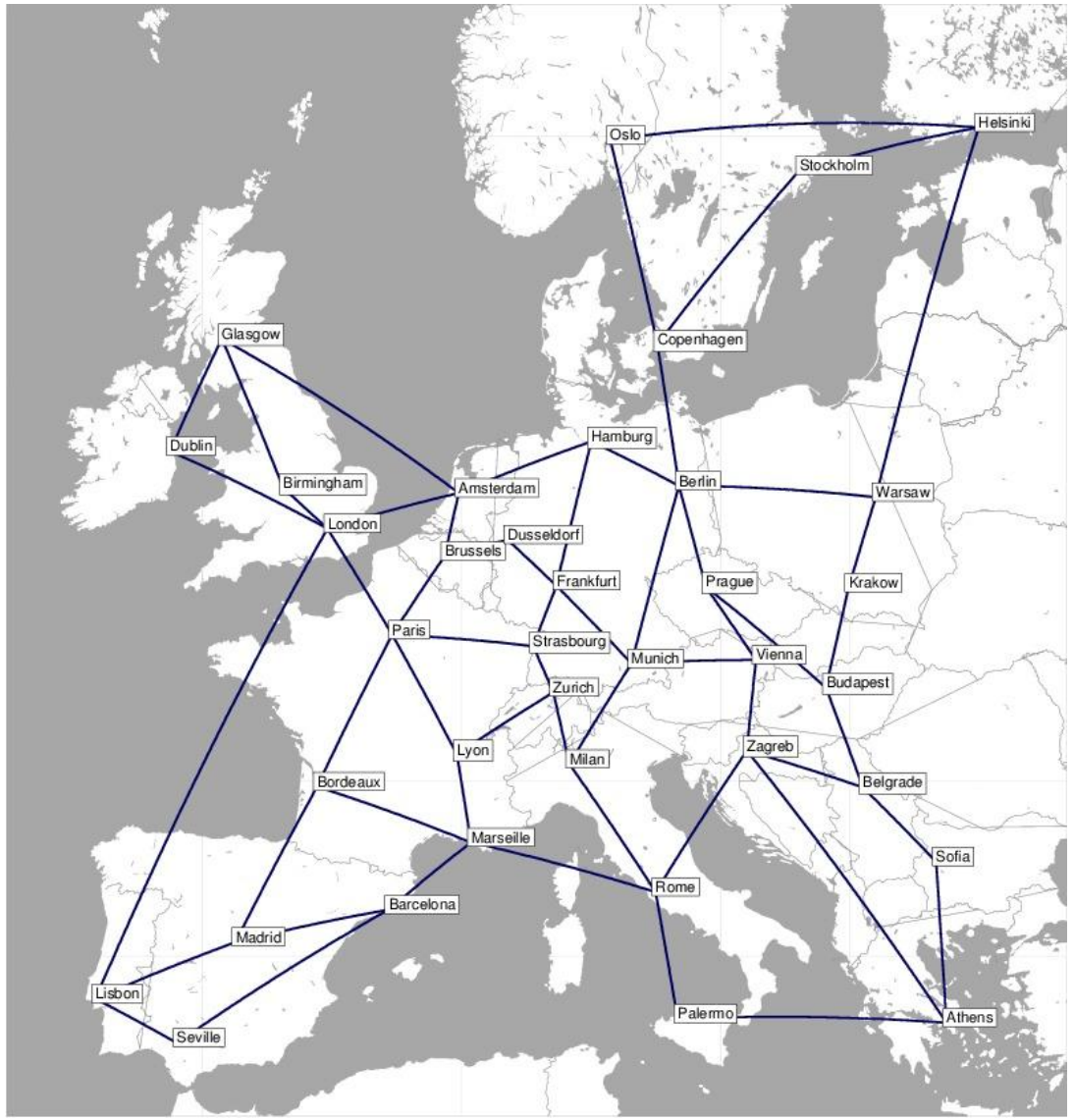
Genetic Algorithm & Integer Programming on Minimization Problem

Problem: Capacitated Network Design Problem (CNDP)

Course: Optimization Methods for Engineers

Authors: Noah Gerber & Krzysztof Łukasik



[1]

1. Introduction: Problem Description

- Given:
  - undirected graph G = (V, UE)
  - $V = \{v \mid v \in \mathbb{N}\}$
  - $UE = \{(ue_i, \{v_1, v_2\}) | ue_i \in N, v_1 \in V, v_2 \in V\}$
  - $R = \{(ue, r_{ue}) \mid \forall ue\ \exists r_{ue},\ ue \in UE,\ r_{ue} \in \mathbb{R}\}$ (routing costs)
  - $M = \{(ue, t, m_{cap}, m_{cost}) \mid \forall ue \forall t\ \exists m_{cap} \exists m_{cost},\ ue \in UE,\ t \in \{1,2,3\},\ m_{cap} \in \mathbb{R},\ m_{cost} \in \mathbb{R}\}$ (modules)
  - $D = \{(v_s, v_t, d) \mid v_s \in V, v_t \in V,\ d \in \mathbb{R}\}$ (demands)
- Objective:
  - $\underset{X,Y}{\text{minarg}}((\sum_{\{d \in D\}} \sum_{\{(ue, r_{ue}) \in R\}} x_{d,ue} \cdot r_{ue}) + (\sum_{\{ue \in UE\}} \sum_{\{(ue, t, m_{cap}, m_{cost}) \in M(ue)\}} y_{ue,t} \cdot m_{cost}))$
  - where X is matrix with $x_{d,ue} \in \mathbb{R}$ as columns (flow decision variables)
  - and Y is matrix with $y_{ue,t} \in \{0, 1\}$ as columns (module decision variables)
  - and $M(ue) = \{(ue', t, m_{cap}, m_{cost}) \in M \mid \forall ue', ue' \in UE, ue' = ue\}$ (modules for the edge)
  - and $\forall ue \in UE$: $(\sum_{\{(ue, t, m_{cap}, m_{cost}) \in M(ue)\}} y_{ue,t}) \leq 1$ (constraint: max one module per edge)
  - and $\forall d \forall ue, d \in D, ue \in UE$: $x_{d,ue} \leq \sum_{\{(ue, t, m_{cap}, m_{cost}) \in M(ue)\}} y_{ue,t} \cdot m_{cap}$ (constraint: less flow than capacity per edge)

1. Introduction: Project Objectives

- Investigate how well the Genetic Algorithm (with elitism) performs on optimizing discrete edge modules
- Formulate the problem as an integer program to provide an exact solution baseline
- Investigate the role of Genetic Algorithm parameters on solution quality and runtime
- Assess the performance by comparing Genetic Algorithm results to integer programming method

2. Algorithms: Mixed Integer Linear Programming (MILP)

- extend problem model:
  - let $E = \{(ue_i, v_s, v_t), (ue_i, v_t, v_s) \mid \forall (ue_i, \{v_s, v_t\}) \in UE\}$
  - let $R' = \{(ue_i, v_1, v_2, r_{ue}), (ue_i, v_2, v_1, r_{ue}) \mid \forall \big(ue_i, \{v_1, v_2\}\big\} \in UE\}$
- Decision Variables:
  - $\forall d \forall e \exists z_{d,e} \mid d \in D, e \in E, z_{d,e} \in \mathbb{R}$
  - $\forall \big(ue, t, m_{cap}, m_{cost}\big) \exists y_{ue,t} \mid \big(ue, t, m_{cap}, m_{cost}\big) \in M, y_{ue,t} \in \{0, 1\}$
- Objective Function
  - $\underset{Z,Y}{\text{minarg}}(\left(\sum_{\{d \in D\}} \sum_{\{(ue_i, v_s, v_t, r_{ue}) \in R\prime\}} z_{d,(ue_i, v_s, v_t)} \cdot r_{ue}\right) + \left(\sum_{\{ue \in UE\}} \sum_{\{(ue, t, m_{cap}, m_{cost}) \in M(ue)\}} y_{ue,t} \cdot m_{cost}\right))$
  - where Z is matrix with $z_{d,(ue_i, v_s, v_t)}$ as columns
  - and Y is matrix with $y_{ue,t}$ as columns
- Constraints
  - $\forall ue \in UE: \left(\sum_{\{(ue, t, m_{cap}, m_{cost}) \in M(ue)\}} y_{ue,t}\right) \leq 1$
  - $\forall d \forall (ue_i, v_s, v_t), d \in D, (ue_i, v_s, v_t) \in E, let\ ue = (ue_i, \{v_s, v_t\}): z_{d,(ue_i, v_s, v_t)} \leq \sum_{\{(ue, t, m_{cap}, m_{cost}) \in M(ue\ )\}} y_{ue,t} \cdot m_{cap}$

2. Algorithms: Mixed Integer Linear Programming (MILP)

- MILP:
  - Model definition
  - (Presolve)
  - Relaxation
  - Branch-and-Cut
  - Cutting Planes
- implemented in Python using gurobipy [2]

2. Algorithms: Elitist Genetic Algorithm (GA) - Definitions

- Defining Parameters:
  - npop: population size
  - cxpb: crossover probability
  - mutpb: mutation probability
  - ngen: number of generations
  - penalty_coeff: penalty coefficient
  - max_module_ratio: first generation contains $npop \cdot max_module_ratio$ individuals with maximum-capacity modules only in their genes
  - min_module_ratio: first generation contains $npop \cdot min_module_ratio$ individuals with minimum-capacity modules only in their genes
  - μ: number of individuals to select for next generations
  - λ: number of children to produce each generation
  - indp: independent probability, applied to each gene independently during mutation
  - tournsize: number of individuals randomly selected from population to compete in a tournament

2. Algorithms: Elitist Genetic Algorithm (GA) - Definitions

- Modeling Individuals
  - $I = (mods, flows)$
  - $mods = \left(t^0, \dots, t^{|UE|-1}\right) \mid t^i \in \{0, 1, 2, 3\}$
  - $flows = \{ \left(f_0^d, \dots, f_4^d\right)^T : \forall d \in D, \ \ \forall d' \in D \ (f_i^{d'} \in [0,1] \ and \ \left(\sum_{i=0}^{i=4} f_i^{d'} = 1\right)\}$
  - Note: we precompute 5 shortest paths for each demand and distribute the demand flow onto these 5 shortest paths
- Defining crossover
  - let $I_1 = (mods_1, flows_1)$, $I_2 = (mods_2, flows_2)$
  - $\forall i \ \left(t^i_{mods_1} = cx\left(t^i_{mods_1}, t^i_{mods_2}\right) \ and \ t^i_{mods_2} = cx\left(t^i_{mods_2}, t^i_{mods_1}\right)\right), i \in \{0, \dots, |UE| - 1\}$ (crossover modules)
  - where $cx(a, b)$ chooses uniform randomly between a, b
  - $\forall d \ \left(f^d = cx'\left(f^d, f'^d\right) \ and \ f'^d = cx'\left(f'^d, f^d\right)\right), \ f^d \in flows_1, \ f'^d \in flows_2, d \in D$ (crossover flows)
  - where $cx'(a, b)$ chooses uniform randomly between a, b
  - return modified I_1

2. Algorithms: Elitist Genetic Algorithm (GA) - Definitions

- Defining mutation
  - let $I = (mods, flows)$ be an individual
  - $\forall i \left(t^i_{mods_1} = mut\left(t^i_{mods}\right)\right), i \in \{0, \dots, |UE| - 1\}$
  - $mut(t)\begin{cases} t, \text{with } 1 - indp \text{ probability} \\ t' \in \{0,1,2,3\}, \text{with } indp \text{ probability} \end{cases}$ (t' being chosen uniformly at random)
  - $\forall d \left(f^d = mut'\left(f^d\right)\right), f^d \in flows_1, d \in D$
  - $mut'\left(f^d\right)\begin{cases} f^d, \text{with } 1 - indp \text{ probability} \\ f \in \left\{(f_0, \dots, f_4)^T : f_i \in [0,1], \sum_{i=0}^{i=4} f_i = 1\right\}, \text{with } indp \text{ probability} \end{cases}$ (f being chosen uniformly at random)

2. Algorithms: Elitist Genetic Algorithm (GA) - Definitions

- Defining evaluation
  - let $I = (mods, flows)$ be an individual
  - $\sum_{t^i \in mods} c(t^i) + routing_costs + penalties$ | module setup costs + routing costs
  - $c(t^i) \begin{cases} 0, if\ t = 0 \\ m_{cost}\ where\ (ue_i, t^i, m_{cap}, m_{cost}) \in M, else \end{cases}$ | module of index t^i setup cost for i^{th} undirected edge
  - $routing_costs$ (including penalties):
    - $\sum_{ue \in UE}(rc(ue) \cdot \min(flow(ue), cap(ue)) + \max(0, flow(ue) - cap(ue)) \cdot penalty_coeff)$ | summed routing costs of undirected edges
    - $rc(ue) = r_{ue}$, where $(ue, r_{ue}) \in R$ | routing cost on $ue \in UE$
    - $cap((ue_i, \{v_1, v_2\})) = \begin{cases} 0, if\ t^i = 0, where\ t^i \in mods \\ m_{cost}, \text{where } (ue_i, t^i, m_{cap}, m_{cost}) \in M, \text{where } t^i \in mods \end{cases}$ | chosen module's capacity
    - $flow((ue_i, \{v_1, v_2\})) = flow'(ue_i, v_1, v_2) + flow'(ue_i, v_2, v_1)$ | flow of $(ue_i, \{v_1, v_2\}) \in UE$
    - $flow'(ue_i, v_s, v_t) = \sum_{f^d \in flows} flow''(f^d, ue_i, v_s, v_t)$ | flow of $(ue_i, v_s, v_t) \in E$
    - $flow''(f^{(v_1, v_2, val)}, ue_i, v_s, v_t) = \sum_{i=0}^{i=4} f_i^{(v_1, v_2, val)} \cdot val \cdot ft((v_1, v_2, val), i, (ue_i, v_s, v_t))$ | flow of $(v_1, v_2, val) \in D$ on $(ue_i, v_s, v_t) \in E$
    - $ft(d, i, e) = \begin{cases} 1, if\ path\ i\ of\ demand\ d\ flows\ through\ directed\ edge\ e \\ 0, else \end{cases}$ | indicator

2. Algorithms: Elitist Genetic Algorithm (GA) - Definitions

- Defining 1st Generation individual creation:
  - let $I = (mods, flows)$ be an individual
  - $mods = \begin{cases} \left(\left(t^0, \dots, t^{|UE|-1}\right), where\ \forall i\ \left(t^i = 3\right), i \in \{0, \dots, |UE|-1\}\right)\ ,\ with\ maxModuleRatio\ probability \\ \left(\left(t^0, \dots, t^{|UE|-1}\right), where\ \forall i\ \left(t^i = 1\right), i \in \{0, \dots, |UE|-1\}\right)\ ,\ with\ minModuleRatio\ probability \\ \left(\left(t^0, \dots, t^{|UE|-1}\right), where\ \forall i\ \left(t^i \in \{0,1,2,3\}\right), i \in \{0, \dots, |UE|-1\}\right)\ , else \end{cases}$
  - $flows = \{\ \left(f_0^d, \dots, f_4^d\right)^T : \forall d \in D,\ \ \forall d' \in D\ \left(f_i^{d'} \in [0,1]\ and\ \left(\sum_{i=0}^{i=4} f_i^{d'} = 1\right)\right\}$ (random flow distribution)

2. Algorithms: Elitist Genetic Algorithm (GA)

- Elitist Genetic Algorithm: [4][5]
  1) create 1st generation with $npop$ individuals
  2) $P_{parent} =$ 1st generation
  3) evaluate P_{parent}
  4) create next generation
     4.1) choose operation: crossover, mutation, reproduction
        - crossover is run with cloned $I_1 \in P_{parent}, I_2 \in P_{parent}$ chosen uniformly at random, returned individual is added to $P_{offspring}$
        - mutation is run with cloned $I \in P_{parent}$ chosen uniformly at random, returned individual is added to $P_{offspring}$
        - reproduction adds a cloned $I \in P_{parent}$ chosen uniformly at random to $P_{offspring}$
     4.4) repeat 4.1) until $|P_{offspring}| = \lambda$
     4.5) evaluate $P_{offspring}$
     4.6) select μ individuals from $(P_{parent} \cup P_{offspring})$, that's new P_{parent}
  5) repeat 2) $ngen$ times
  6) Result: $\min(\{evaluate(p) : p \in P_{parent}\})$
- Implemented in Python using DEAP [2]

3. Parametric Optimization – Fixed Parameters

	npop	max-module-ratio	min-module-ratio	μ	μ	ngen
Description	Initial population size	initial population % with maximum capacity modules selected	initial population % with minimum capacity modules selected	number of individuals selected per gen	number of offspring created each gen	Number of generations
Value	80	0.4	0.2	80	136	40

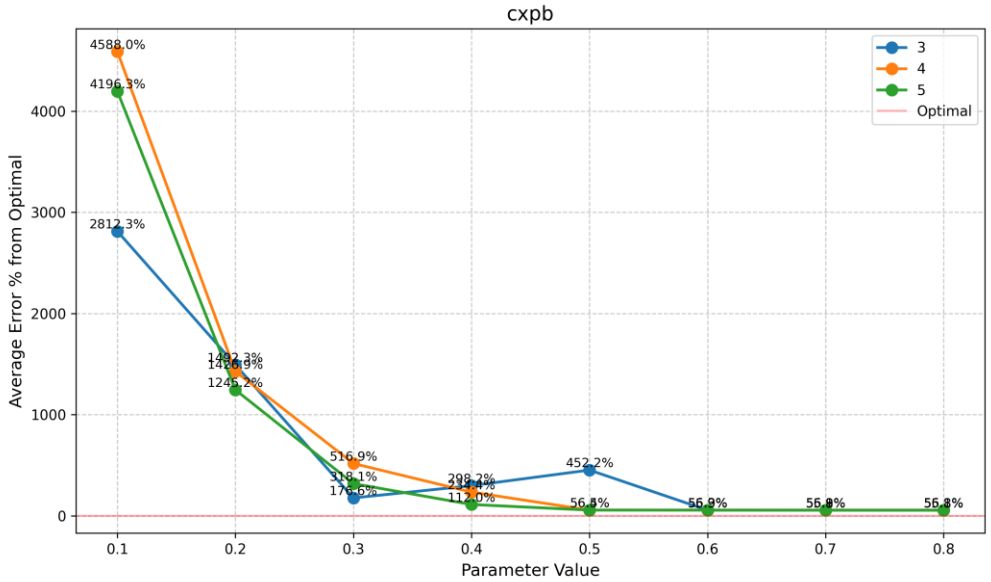
3. Parametric Optimization- Variable Parameters

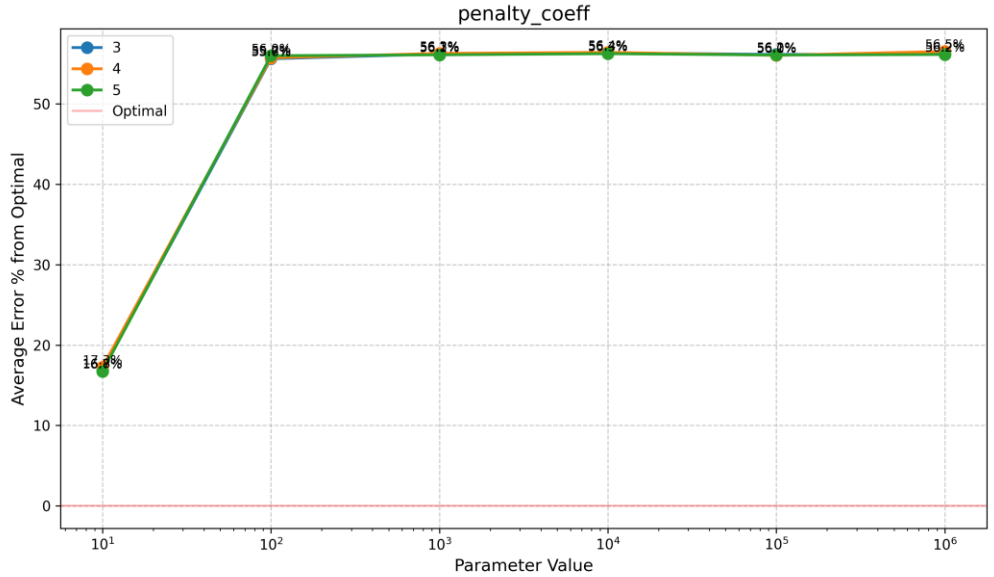
	cxpb	mutpb	penalty-coeff	indpb	tournsize
Description	Cross over probability	Mutation probability	Penalty for each flow unit over the capacity of an edge	mutation probability for each $t^i \in mods$ and each $f^d \in flows$	Tournament Size
Values	$\{0.1 \cdot k : k \in \{0, \dots, 8\}\}$	$\{0.1 \cdot k : k \in \{0, \dots, 4\}\}$	$\{10^k : k \in \{1, \dots, 6\}\}$	$\{0.1 \cdot k : k \in \{0, \dots, 4\}\}$	$\{2, 3, 4, 5\}$
Default	0.6	0.2	100000	0.4	3

3. Parametric Optimization- Approach

- for each value of each variable parameter, the algorithm was run multiple times
- average runtime and average cost were calculated from running the algorithm 3, 4 and 5 times for each value
- as benchmark runtime and cost we took the best result returned by the integer programming algorithm
- the integer programming algorithm was executed with an imposed computational time limit of 180 seconds

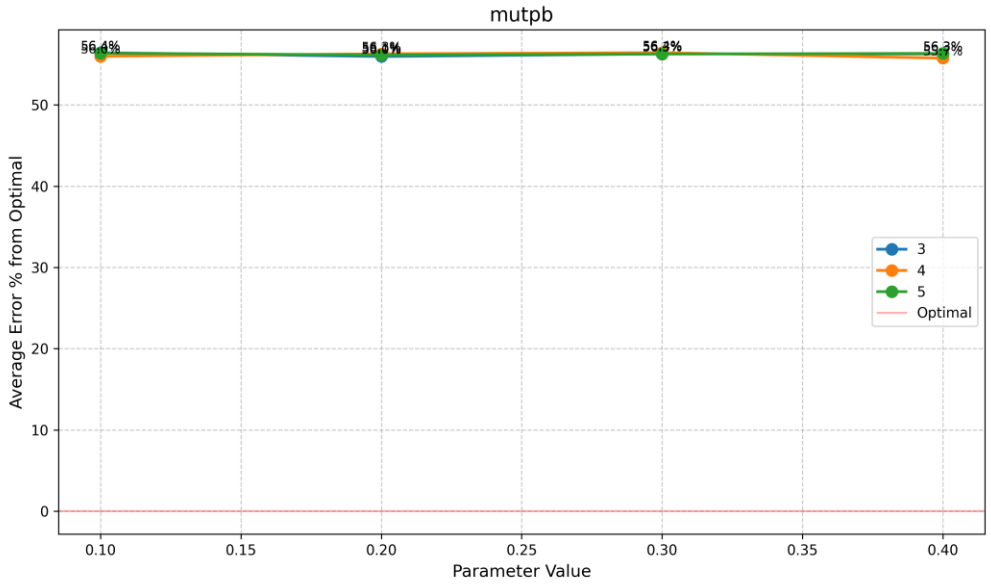
3. Parametric Optimization – Results (cost)

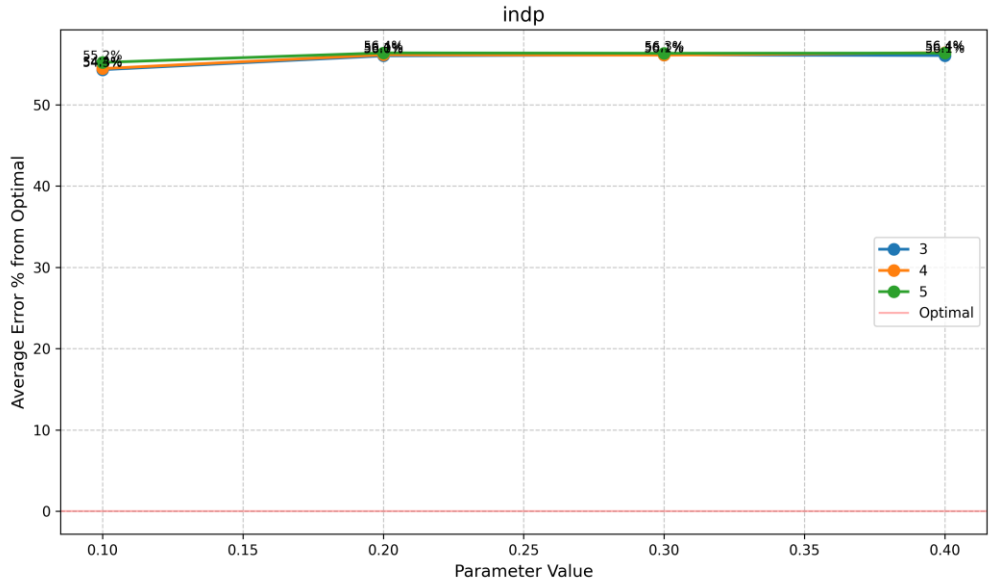




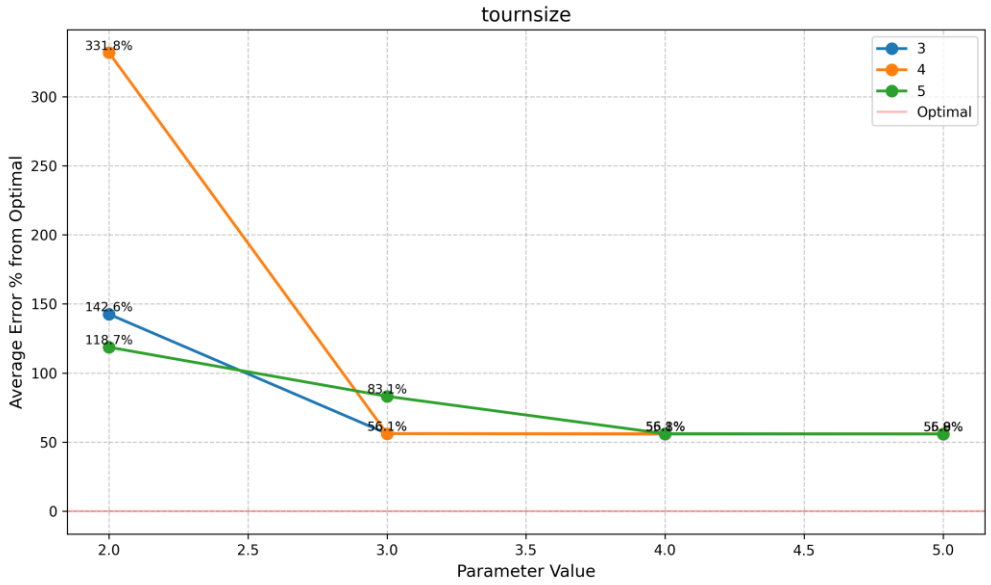
- Observations
  - $cxpb > 0.5 \quad \leftrightarrow 0\ penalties$
  - $penaltyCoeff > 10 \ \leftrightarrow 0\ penalties$
- note: penalties are applied when there is more flow on an edge than the selected modules capacity, therefore penalties indicate invalid solutions

3. Parametric Optimization – Results (cost)



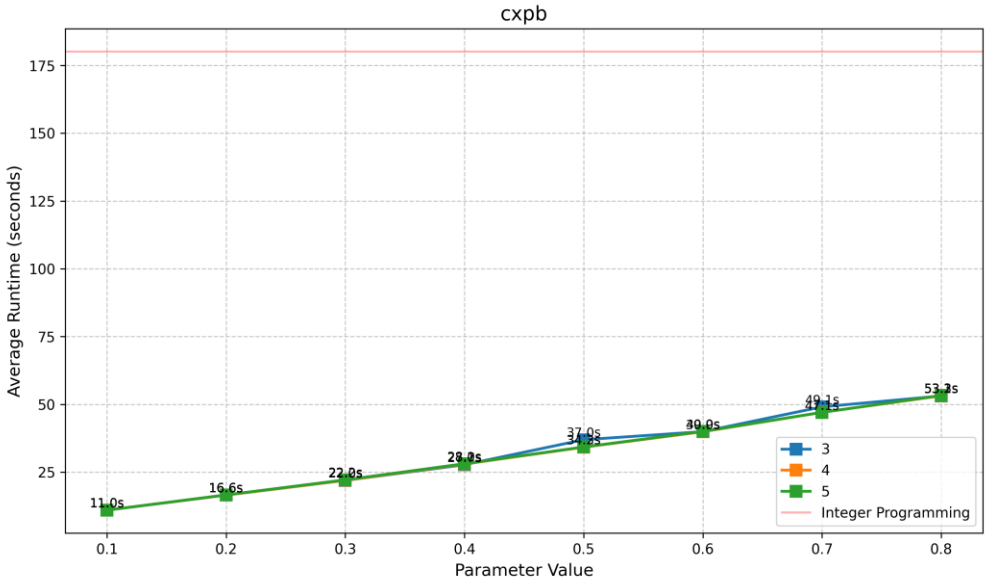


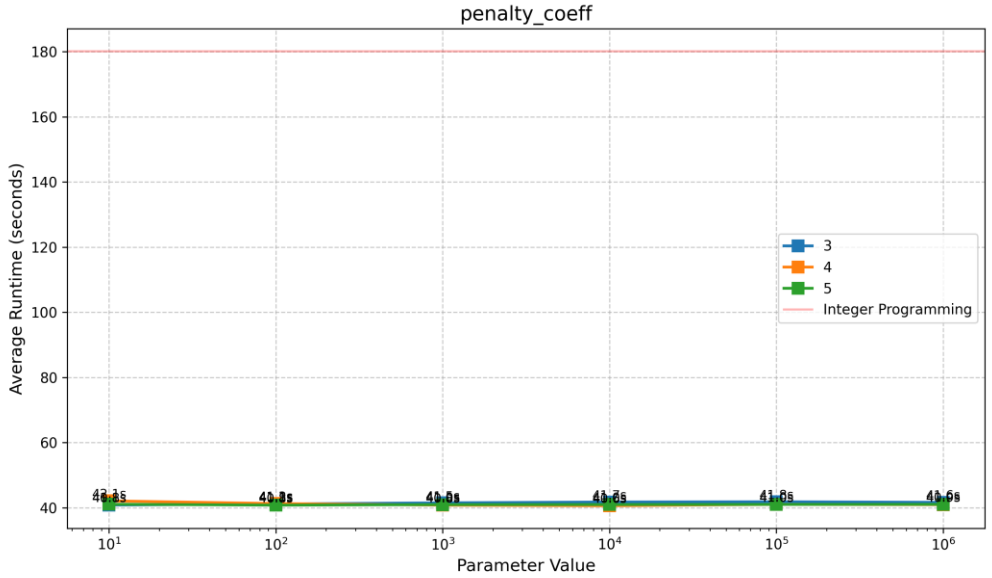
3. Parametric Optimization – Results (cost)



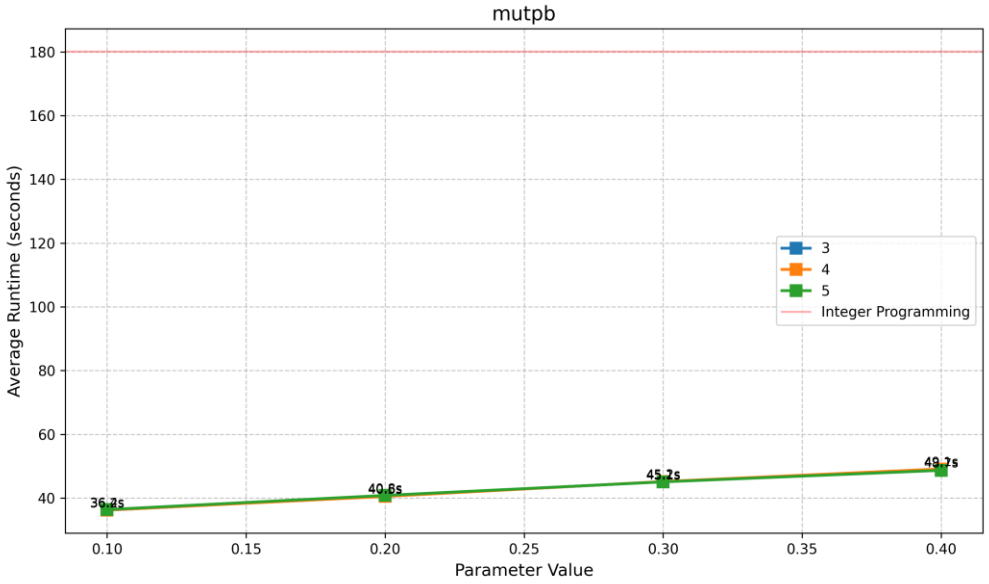
- Observation:
  - $tournsize > 2 \leftrightarrow 0\ penalties$

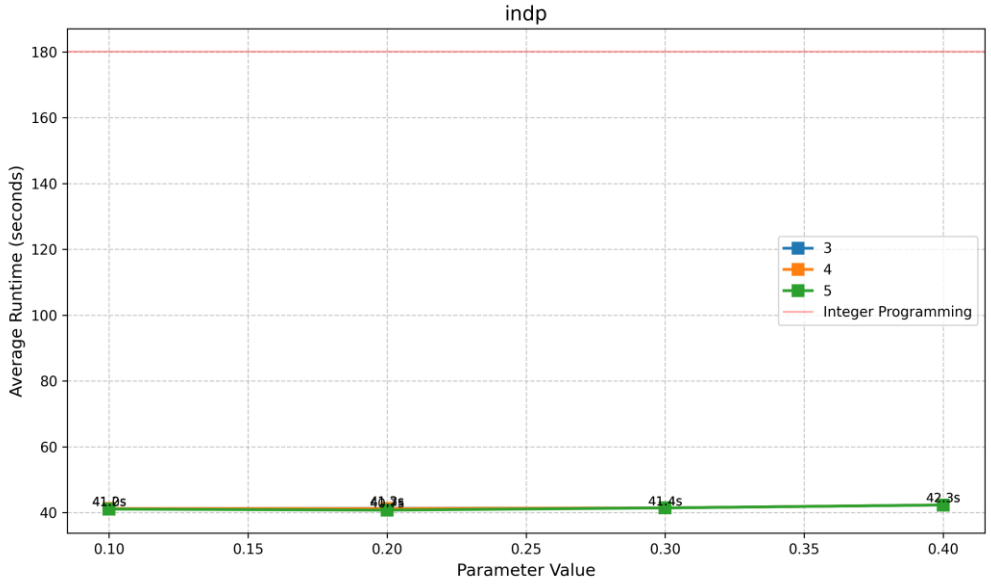
3. Parametric Optimization – Results (runtime)



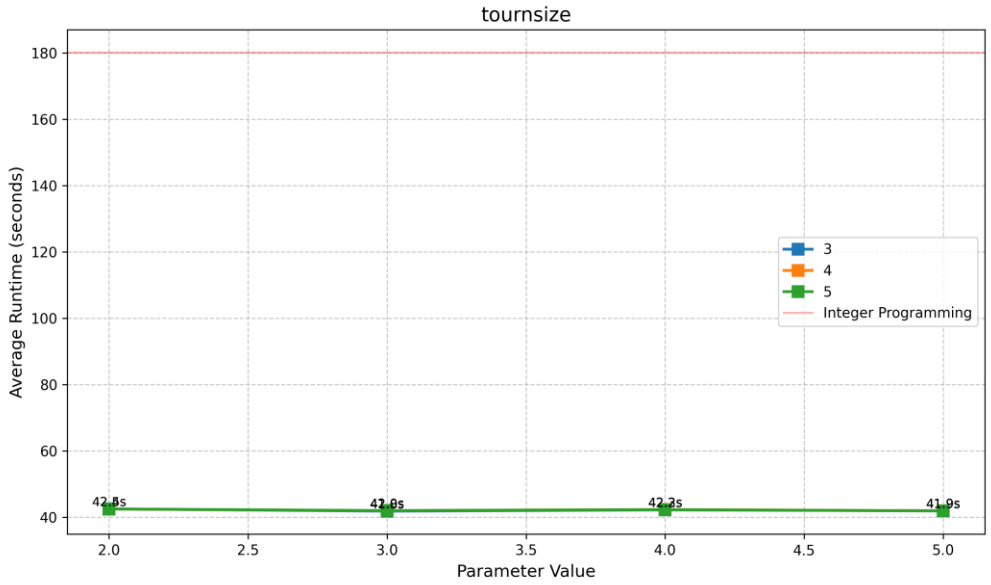


3. Parametric Optimization – Results (runtime)





3. Parametric Optimization – Results (runtime)



4. Results – Chosen Parameters

	npop	ngen	cxpb	penalty_coeff	μ	λ
Value	80	40	0.8	100000	80	136

	max_module_ratio	min_module_ratio	mutpb	indp	tournsize
Value	0.4	0.2	0.2	0.4	3

4. Results - Approach

- parameters yielding the best results were picked
- as benchmark runtime and cost we took the best result returned by the integer programming algorithm
- the integer programming algorithm was executed with an imposed computational time limit of 180 seconds
- the Genetic Algorithm was run 10 times, the average of the returned costs and runtimes was calculated

	Integer Programming Algorithm	Elitist Genetic Algorithm
cost	$2.995 \cdot 10^7$	$4.667 \cdot 10^7$
runtime [s]	180	53.77
error (from IP) [%]	0	55.79

5. Conclusions & Discussion

- Integer Programming
  - High-quality, deterministic solutions
  - Long runtime, limited scalability
- Genetic Algorithm
  - Faster, flexible, scalable
  - Stochastic, less consistent quality
  - less precise

Conclusion:
Trade-offs exist between runtime efficiency and solution quality. Choice depends on application requirements: IP for precision, GA for speed and scalability. Hybrid approaches may leverage strengths of both

References

[1] P. Audenaert, “Network cost266”. SNDlib. [Online]. Available: https://sndlib.put.poznan.pl/home.action.
[Accessed: Jul. 24, 2025]

[2] Gurobi Optimization LLC, “gurobipy 12.0.3”, PyPi. [Online]. Available: https://pypi.org/project/gurobipy/.
[Accessed: Jul. 24, 2025]

[3] Unknown Author, “Algorithms”, Read the Docs. [Online]. Available: https://deap.readthedocs.io/en/master/api/algo.html#deap.algorithms.eaMuPlusLambda.
[Accessed: Jul. 24, 2025]

[4] Unknown Author, “Algorithms”, Read the Docs. [Online]. Available: https://deap.readthedocs.io/en/master/api/algo.html#deap.algorithms.varOr.
[Accessed: Jul. 24, 2025]

[5] F.-A. Fortin, F.-M. De Rainville, M.-A. Gardner, M. Parizeau, and C. Gagné, "DEAP Documentation," Read the Docs. [Online]. Available: https://deap.readthedocs.io/en/master/.
[Accessed: Jul. 24, 2025]